Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Dersin tarihi 20 Kasım 2020 Cuma. Ders Zoom üzerinden yapıldı.
  - Asıl anlatım yaklaşık 00:11:40'ta başlar. Kaydın başı vize duyuruları ve Zoom'daki bir isim karışıklığıdır.
  - Ders bir arayla iki bölüm. Ara yaklaşık 01:07-01:16 arasıdır.
  - Bu ders vizeden önceki son derstir. Vize bir sonraki Cuma (8. hafta) yapıldı.
- Algoritmalar bölümünün devamı: temel veri yapıları
  - Dizi (array), bağlı liste (linked list).
  - Kuyruk (queue) ve yığın (stack); push, pop ve yığın göstergesi (stack pointer).
- Yığın uygulaması: matematiksel ifadelerin gösterimi
  - Ara ek (infix), ön ek (prefix, Polish notation) ve son ek (postfix, Reverse Polish notation) gösterimleri.
  - İfadeyi ikili ağaca (binary tree) yerleştirme ve buradan prefix/postfix yazma.
  - Yığın ile infix, prefix ve postfix ifadelerin değerlendirilmesi.
  - Gösterimler arası dönüşümler: postfix → infix, prefix → postfix.
- Algoritma analizi
  - Doğruluk, zaman verimliliği, bellek kullanımı ve optimallik.
  - Teorik analiz ve deneysel (kaba) analiz.
  - Temel işlem (basic operation) ve giriş boyutu (input size).
  - Yürütme zamanı (running time) $T(n)$, alan maliyeti (space cost), zaman ve alan karmaşıklığı.
  - Kod üzerinden $T(n)$ hesabı: ortalama bulma, en küçük elemanı bulma, seçmeli sıralama (selection sort), matris toplama.
- Fonksiyonların büyümesi ve asimptotik gösterim
  - Büyüme sırası: $1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$.
  - En kötü durum (worst case), en iyi durum (best case), ortalama durum (average case).
  - Büyük O (Big-O), Büyük Omega (Big-Omega) ve Büyük Teta (Big-Theta) tanımları ve tanık (witness) sabitleri.
  - Birden fazla alt programdan oluşan algoritmanın karmaşıklığı: toplamda en büyüğü, çarpımda çarpım.
  - Yinelemeli (iterative) algoritmaların analizi: araya ekleme sıralaması (insertion sort) ve seçmeli sıralama.
- Özyinelemeli (recursive) algoritmaların analizine geçilmedi. Hoca bu konunun vizeye dahil olmadığını söyledi.

Hocanın Özellikle Vurguladığı Kısımlar

- Vize hakkında:
  - Vize bir sonraki Cuma (27 Kasım 2020). Hoca saatin değişmiş olabileceğini, sınav programından bakılmasını söyledi.
  - Büyük ihtimalle klasik sınav olacak. Klasik olursa yaklaşık 4 soru sorulacak.
  - Sınav, hocanın bu derste geldiği yere kadar olan konuları kapsar. Özyinelemeli algoritmaların analizi ve rekürans bağıntıları dahil değil.
- Klasik sınav kâğıdının fotoğrafını düzgün çekin.

  - Kâğıt düz bir zeminde, tam yukarıdan, kadraja tam girecek şekilde çekilsin. Işığa dikkat edin.
  - Arkada yatak, çarşaf gibi ev eşyaları görünmesin. Kâğıt yan (90°) gelmesin.
  - Önceden deneme yapın. Bir öğrenci kâğıdı tarayıp PDF yapan Office Lens uygulamasını önerdi; hoca bunun bütün sınavlar için faydalı olduğunu söyledi.
- Bu derste görülen her şeyin en az %80'ini ileriki derslerde ve kod yazarken farkında olmadan kullanacaksınız.
  - Veri yapılarının ve algoritma analizinin ayrıntıları Veri Yapıları ve Algoritma Analizi derslerinde gelecek.
- Prefix ve postfix yazarken işlem sırasını mutlaka soldan sağa takip edin.
  - Aynı önceliğe sahip işlemler (çarpma ve bölme, toplama ve çıkarma) soldan sağa yapılır.
  - Prefix ve postfix ifadelerde parantez yoktur. Sıra, işaretlerin yerinden anlaşılır.
- Üs alma işleminde operandların sırası önemlidir.
  - Prefix'te $\uparrow b\,2$ "b'nin karesi"dir. $\uparrow 2\,b$ ise "2^b"dir.
- Infix değerlendirmede iki yığın, prefix ve postfix değerlendirmede tek yığın kullanılır.
  - Bu yüzden derleyiciler ifadeleri arka planda prefix veya postfix biçime çevirerek daha hızlı işler.
- Algoritma sorusunda sizden kod istenmeyecek. Yığın üzerinde işlemin nasıl yürüdüğünü bilmeniz yeter.
- $T(n)$ hesabında her işlemin en kötü durumda kaç kez yapıldığını sayın.
  - Bir `if` bloğunun kaç kez çalışacağı bilinmiyorsa, her seferinde çalışıyormuş gibi alın.
- Karmaşıklık bulurken sabitler ve düşük dereceli terimler atılır. En yüksek dereceli terim kalır.
  - n ile $n-1$ arasındaki fark, $n = 10\,000$ gibi büyük değerlerde önemsizdir.
  - İç içe iki döngü varsa genelde $O(n^2)$, üç döngü varsa $O(n^3)$ denir.
- Biz her zaman en kötü duruma (Big-O) göre çalışırız. Algoritma kitapları da genelde Big-O verir.
- İki algoritmadan küçük girişlerde hızlı olanı değil, büyük girişlerde daha yavaş büyüyeni seçin.
  - Örnek: $T_A(n) = 50n$, $T_B(n) = n^2$. $n < 50$ için B hızlı, ama girişin büyümeyeceği garanti değilse A seçilir.
- Algoritmik düşünmeyi iyi öğrenin.
  - Gördüğünüz algoritmaların kodunu kendiniz yazın, analiz edin, kitaptaki karmaşıklıkla karşılaştırın.
  - "Başkasıyla değil, kendinle yarış."
- Hoca slaytlarda hata olabileceğini, görürseniz söylemenizi istedi. Seçmeli sıralama slaytındaki hatayı bir öğrenci yakaladı.

Kısa Tekrar Notları

- Dizi: bellekte ardışık gözler. Elemana sırayla da rastgele (random) de erişilir.
- Bağlı liste: her göz bir veri ve bir sonraki gözün adresini (link) tutar. Ekleme ve silmede yalnız linkler değişir.
- Kuyruk: ilk giren ilk çıkar, FIFO (first in, first out). Ekleme sondan, çıkarma baştan yapılır.
- Yığın: son giren ilk çıkar, LIFO (last in, first out). Push ekler, pop çıkarır. Yığın taşması: stack overflow.
- Infix: $A + B$. Prefix: $+AB$. Postfix: $AB+$.
- İşlem önceliği: parantez, üs, çarpma/bölme, toplama/çıkarma. Aynı öncelikte soldan sağa.
- Yürütme zamanı $T(n)$: algoritmanın yaptığı temel işlemlerin sayısını n'e bağlı veren ifade.
- `for (k=0; k<n; k++)`: atama 1, karşılaştırma $n+1$, artırma n kez. Toplam $2n+2$.
- Büyüme sırası: $1, \log n, n, n \log n, n^2, n^3, 2^n, n!$.
- $f(x) = O(g(x))$: $x > k$ için $|f(x)| \leq C|g(x)|$ olacak şekilde C ve k sabitleri (tanıklar) vardır.
- $f(x) = \Omega(g(x))$: $x > k$ için $|f(x)| \geq C|g(x)|$.
- $f(x) = \Theta(g(x))$: hem $O(g(x))$ hem $\Omega(g(x))$.
- Toplam kuralı: $O(f) + O(g) = O(\max(f, g))$. Çarpım kuralı: $O(f) \cdot O(g) = O(f \cdot g)$.
- Sıralı arama: en iyi 1, en kötü n, ortalama $(n+1)/2$ karşılaştırma.
- İkili arama $O(\log n)$. Kabarcık, araya ekleme ve seçmeli sıralama en kötü durumda $O(n^2)$.

- Hanoi kuleleri $O(2^n)$.

Detaylı Açıklamalar

Temel veri yapıları

Veri yapısı, veriyi tuttuğumuz ve özelliklerine göre işlediğimiz yapıdır. Kümeler, diziler, matrisler, bağlı listeler, yığınlar, kuyruklar, ağaçlar, graflar ve hash tabloları birer veri yapısıdır.

Bağlı liste örneği: Dizi $5, 7, 1, 8, 4$ (indisler 1-5) bağlı liste mantığıyla küçükten büyüğe sıralanıyor.

İndis	Veri	Link
1	5	2
2	7	4
3	1	5
4	8	dur
5	4	1

- Başlangıç (start) 3 numaralı gözdür (en küçük eleman 1).
- Gezinti: $1 \to 4 \to 5 \to 7 \to 8$.
- **6'yı ekleme:** Baştan gezilir. 6, 7'den küçük olduğu için 5 ile 7 arasına girer. 6 numaralı göze yazılır. 5'in linki 6 olur, 6'nın linki eski değer olan 2 olur.

Kuyruk: Bilet veya havaalanı check-in sırası gibidir. İlk gelen işini bitirip ilk çıkar.

Yığın: Bir ucu kapalı kavanoz gibidir. En üstteki eleman çıkmadan alttaki çıkamaz. - Bir diziyi ters çevirmek için elemanları sırayla push edip sonra pop etmek yeter: $1, 2, 3, 4, 5 \to 5, 4, 3, 2, 1$. - Yığın göstergesi (SP) yığının tepesini gösterir. Her push ve pop'ta eleman boyutu kadar (1 bayt veya 2 bayt) değişir. - Yığın, Alt Seviye Programlama dersinde register ve bellek işlemlerinde sık karşınıza çıkacak.

Prefix, infix ve postfix gösterimleri

İfadeyi ikili ağaca yerleştirmek dönüşümü kolaylaştırır. Kökte en son yapılacak işlem durur. - Ağacın ön sıra (preorder) gezintisi prefix, son sıra (postorder) gezintisi postfix ifadeyi verir.

Örnek 1: $(A+B)*(C/D)$ (ağaç kareden doğrulandı) - Kök $*$, sol alt ağaç $A+B$, sağ alt ağaç C/D. - Prefix: $* + A\,B\,/\,C\,D$ - Postfix: $A\,B + C\,D/*$

Örnek 2: $A * B/C + (C/D - A)$ - İşlem sırası: $A * B$, sonuç $/C$; C/D, sonuçtan A çıkar; iki sonucu topla. - Prefix: $+\,/ * A\,B\,C\,-/C\,D\,A$ - Postfix: $A\,B * C\,/\,C\,D/A - +$ - Uyarı: $A\,B*$'dan sonra C'yi yanlış yere koyarsanız "C'yi $A * B$'ye böl" anlamı çıkar.

Örnek 3: $\dfrac{b^2 - 4ac}{2a}$ (ağaç ve sonuçlar kareden doğrulandı) - Kök $/$. Sol: $b^2 - 4ac$, sağ: $2a$. - Prefix: $/ - \uparrow b\,2\ **4\,a\,c\ * \ 2\,a$ - Postfix: $b\,2\uparrow\ 4\,a * c * \ - \,2\,a * \ /$ - b^2 yerine $*\,b\,b$ de yazılabilir. Ama yüksek kuvvetlerde üs işareti kullanmak daha doğrudur.

Örnek 4 (sayısal): $(3+2)^2 + 8/4 * 3$ - Prefix: $+ \uparrow +3\,2\,2\ */8\,4\,3$ - Postfix: $3\,2+2\uparrow\ 8\,4/3*\ +$ - Değer: $5^2 = 25$, $8/4 = 2$, $2 \cdot 3 = 6$, $25 + 6 = 31$. Üç gösterim de aynı sonucu verir.

Yığın ile ifade değerlendirme

Infix ifade: $2 * (5 * (3 + 6))/15 - 2$ (slayt 32, kareden doğrulandı) - İki yığın vardır: operand yığını ve operatör yığını. - Sayılar operand yığınına, işaretler ve açma parantezleri operatör yığınına atılır. - Kapama parantezi gelince, açma parantezine kadar olan işlemler yapılır: - $3+6 = 9$ push edilir. Sonra $9 \cdot 5 = 45$ push edilir. - Yeni bir operatör gelince yığının tepesindeki operatörle öncelik karşılaştırılır. Tepedeki daha öncelikliyse önce o işlem yapılır. - $-$ gelince $/$ yapılır: $45/15 = 3$. Sonra $*$ yapılır: $3 \cdot 2 = 6$. - Sonda kalan işlem: $6 - 2 = 4$. Sonuç **4**.

Prefix ifade: $- / * \; 20 * 50 + 3\,6 \; 300 \; 2$ (slayt 34) - Önce ifade ters çevrilir, sonra soldan okunur. Tek yığın yeterlidir. - Sayı gelince push edilir. İşaret gelince iki sayı pop edilir, sonuç push edilir. - $3 + 6 = 9$; $50 \cdot 9 = 450$; $20 \cdot 450 = 9000$; $9000/300 = 30$; $30 - 2 = 28$. - Sonuç **28**.

Postfix ifade: $2\,5\,3\,6 + * * 5 / 2 -$ (slayt 36) - Soldan okunur. Sayı push edilir. İşaret gelince iki sayı pop edilir. - $3 + 6 = 9$; $5 \cdot 9 = 45$; $2 \cdot 45 = 90$; $90/5 = 18$; $18 - 2 = 16$. - Sonuç **16**.

Dönüşümler: - Postfix → infix: İşaretler soldan sağa okunur. Her işaret, önündeki iki ifadeyi birleştirir. Örnek: $B\,C\,/$ "B/C" olur, sonra A'dan çıkarılır. - Prefix → postfix: Aynı mantıkla alt ifadeler bulunur, işaret sona taşınır. - Slaytlarda bu dönüşümlerin yığınla adım adım yapılışı var. Hoca bunları kendi başınıza izlemenizi istedi.

Algoritma analizi

Bir algoritmada doğruluk, zaman verimliliği, bellek kullanımı ve optimallik önemlidir. - **Teorik analiz:** Zaman, giriş boyutuna bağlı olarak temel işlem sayısıyla ölçülür. - **Deneysel analiz:** Bir giriş seçilir, algoritma çalıştırılır, süre ölçülür.

Problem	Giriş boyutu	Temel işlem
n elemanlı dizide arama	n	Karşılaştırma
İki matrisi çarpma	Matris boyutu	Çarpma
Bir sayının asal olup olmadığını bulma	Sayının ikili basamak sayısı	Bölme (mod)
Graf problemi	Düğüm ve kenar sayısı	Düğüm/kenar gezme

- **Yürütme zamanı $T(n)$:** Algoritmanın işini yapması için kaç temel işlem gerektiğini n'e bağlı veren ifade.
- **Alan maliyeti:** Algoritmanın gerektirdiği bellek. Örneğin yığın taşıyorsa daha büyük yığın gerekir.
- **Zaman ve alan karmaşıklığı:** n büyürken yürütme zamanının ve bellek ihtiyacının nasıl arttığını gösteren asimptotik ifadeler.

$T(n)$ hesabı örnekleri

Örnek 1: Ortalama bulma (slayt kareden doğrulandı)

```
float BulOrta(float A[], int n) {
    float ortalama, toplam = 0;   // 1 işlem
    int k;
    for (k = 0; k < n; k++)       // 1 + (n+1) + n = 2n+2
        toplam = toplam + A[k];   // 2 işlem, n kez: 2n
    ortalama = toplam / n;        // 2 işlem
    return ortalama;              // 1 işlem
}
```

- $T(n) = 1 + (2n + 2) + 2n + 2 + 1 = 4n + 6$

Örnek 2: En küçük elemanı bulma (slayt kareden doğrulandı)

```
float BulEnkucuk(float A[]) {
    float enkucuk;
    int k;
    enkucuk = A[0];               // 1
    for (k = 1; k < n; k++)       // 1 + n + (n-1) = 2n
        if (A[k] < enkucuk)       // n-1
            enkucuk = A[k];       // en kötü durumda n-1
    return enkucuk;               // 1
}
```

- `if` bloğunun kaç kez çalışacağı belli değil. En kötü durumda her seferinde çalışır.
- $T(n) = 1 + 2n + (n - 1) + (n - 1) + 1 = 4n$

Örnek 3: Seçmeli sıralama (slayt kareden doğrulandı)

```
public static int[] selectionsort(int[] A, int n) {
    int tmp; int min;
    for (int i = 0; i < n-1; i++) {          // I
        min = i;                             // II
        for (int j = i; j < n; j++) {        // III
            if (A[j] < A[min]) {             // IV
                min = j;                     // V
            }
        }
        tmp = A[i]; A[i] = A[min]; A[min] = tmp;   // VI
    }
    return A;                                // VII
}
```

Satır	İşlem	Tekrar	Toplam
I	1, 1, 1	$1, n, n$	$2n+1$
II	1	n	n
III	1, 1, 1	n, $\frac{n(n+1)}{2}$, $\frac{n(n+1)}{2}$	$n+n(n+1)$
IV	1	$\frac{n(n+1)}{2}$	$\frac{n(n+1)}{2}$
V	1	$\frac{n(n+1)}{2}$	$\frac{n(n+1)}{2}$
VI	1, 1, 1	n	$3n$
VII	1	1	1

- İç döngü $j = i$'den başladığı için her turda bir azalarak döner: $n, n-1, \ldots, 1$. Toplamı $\frac{n(n+1)}{2}$'dir.
  - Hoca bunu $n = 5$ ile gösterdi: $5+4+3+2+1 = 15 = \frac{5 \cdot 6}{2}$.
- VI satırı dış döngünün içinde olduğu için n kez çalışır. Slaytta önce 3 yazıyordu. Bir öğrenci hatayı fark etti, hoca $3n$ olarak düzeltti.
- Düzeltilmiş sonuç: $T(n) = 2n^2 + 9n + 2$. Karmaşıklık $O(n^2)$.

Örnek 4: Matris toplama ($n \times m$) - Dış döngü: $1 + (n+1) + n = 2n + 2$. - İç döngü: $(1 + (m+1) + m) \cdot n = 2mn + 2n$. - Toplama ve atama: $2mn$. - Toplam: $4mn + 4n + 2$. $m = n$ alınırsa $T(n) = 4n^2 + 4n + 2$, karmaşıklık $O(n^2)$.

Fonksiyonların büyümesi

- Karmaşıklık, veri kümesinin eleman sayısı arttıkça yürütme zamanının nasıl değiştiğini gösterir.
- n, 10'dan 10^6'ya çıkarken $\log n$ çok az, n doğrusal, n^2 hızla, 2^n ve $n!$ hesaplanamayacak kadar büyür.
- Bilinen algoritmalar:
  - Sabit $O(1)$: diziye eleman eklemek gibi n'e bağlı olmayan işlem.
  - Logaritmik $O(\log n)$: ikili arama. İşlem sürekli yarıya bölünüyorsa işin içinde logaritma vardır.
  - Doğrusal $O(n)$: doğrusal (sıralı) arama.
  - $O(n \log n)$: böl ve fethet (divide and conquer) algoritmaları, örneğin quicksort.
  - Karesel $O(n^2)$: kabarcık, araya ekleme ve seçmeli sıralama.
  - Kübik $O(n^3)$: iki $n \times n$ matrisin klasik çarpımı, üç iç içe döngü.
  - Üstel $O(2^n)$: Hanoi kuleleri.
  - Faktöriyel $O(n!)$.
- Kaba kuvvet (brute force) algoritmalar, olabilecek bütün durumları dener.

En kötü, en iyi ve ortalama durum

- **En kötü durum:** Algoritmanın en uzun sürdüğü giriş. Çalışma zamanına bir üst sınır ve garanti verir.
  - Aramada aranan eleman dizide yoksa döngü n kez çalışır.
- **En iyi durum:** Algoritmanın en kısa sürdüğü giriş. Aranan eleman ilk sırada çıkarsa. Pek dikkate alınmaz.
- **Ortalama durum:** Bütün girişlere göre ortalama. En iyi ve en kötünün ortalaması değildir; ayrı hesaplanır.
- **Sıralı arama:**
  - En iyi: 1 karşılaştırma. En kötü: n.
  - Ortalama: eleman $1., 2., \dots, n.$ sırada eşit olasılıkla bulunursa $\frac{1+2+\cdots+n}{n} = \frac{n+1}{2}$.

Asimptotik gösterimler

Big-O: f ve g, tam sayılardan veya reel sayılardan reel sayılara fonksiyonlar olsun. $x > k$ için $|f(x)| \leq C|g(x)|$ sağlayan C ve k sabitleri varsa $f(x) = O(g(x))$ yazılır. C ve k'ye tanık (witness) denir. - Hocanın benzetmesi: g, 10 ders alan birinin hazırlık süresi; f, 7 ders alan birinin. 7 ders alan kişi en kötü ihtimalle o süre kadar harcar.

Örnek: $f(x) = x^2 + 2x + 1$ için $O(x^2)$. - $k = 1$: $x > 1$ ise $x < x^2$ ve $1 < x^2$. Buradan $x^2 + 2x + 1 \leq x^2 + 2x^2 + x^2 = 4x^2$. Tanıklar $C = 4$, $k = 1$. - $k = 2$: $x > 2$ ise $2x < x^2$ ve $1 < x^2$ (bu yüzden $2x + 1 \leq 2x^2$). Buradan $x^2 + 2x + 1 \leq 3x^2$. Tanıklar $C = 3$, $k = 2$. - Kontrol: $x = 3$ için $16 \leq 27$.

Big-Omega örneği (slayt kareden doğrulandı): $f(x) = 8x^3 + 5x^2 + 7 \geq 8x^3$, bütün pozitif x için. Yani $f(x) = \Omega(x^3)$.

Big-Theta: $x > k$ için $C_1|g(x)| \leq |f(x)| \leq C_2|g(x)|$ sağlanırsa $f(x) = \Theta(g(x))$.

Limit ile karşılaştırma: $\lim_{x\to\infty} \frac{4x^3 + 3x^2}{x^4} = 0$. Bu yüzden $4x^3 + 3x^2 = O(x^4)$.

Tümevarım ile: $n \geq 7$ için $3^n < n!$. - Taban: $3^7 = 2187 < 7! = 5040$. - Adım: $3^k < k!$ ise $3^{k+1} = 3 \cdot 3^k < 3 \cdot k! < (k+1) \cdot k! = (k+1)!$. - Sonuç: $3^n = O(n!)$.

Birden fazla parçalı fonksiyonun karmaşıklığı

- Toplanan parçalar için en büyük karmaşıklık alınır. Örnek: $O(\log n) + O(n) = O(n)$; $O(2^n) + O(n) = O(2^n)$.
- Çarpılan parçalar için karmaşıklıklar çarpılır.

Örnek 1 (slayt kareden doğrulandı): $f(n) = 3n\log(n!)+(n^2+3)\log n$ - $3n = O(n)$. - $\log(n!) \leq \log(n^n) = n\log n$. Yani $\log(n!) = O(n\log n)$. - Çarpım: $3n\log(n!) = O(n^2\log n)$. - $(n^2 + 3)\log n = O(n^2\log n)$. - Toplam: $f(n) = O(n^2\log n)$.

Örnek 2 (slayt kareden doğrulandı): $f(x) = (x+1)\log(x^2+1) + 3x^2$ - $x + 1 = O(x)$. - $\log(x^2+1) = O(\log x)$, çünkü $x > 1$ için $\log(x^2+1) \leq \log(2x^2) = \log 2 + 2\log x$. - Çarpım: $O(x\log x)$. $3x^2 = O(x^2)$. - Toplam: $f(x) = O(x^2)$.

Yinelemeli algoritmaların analizi

Araya ekleme sıralaması (insertion sort): Dizi $11, 7, 15, 3, 16, 13$. - İkinci elemandan başlanır. Eleman bir anahtara (key) alınır. Solundaki kendisinden büyük elemanlar birer sağa kaydırılır. Açılan yere anahtar yazılır.

Adım	Anahtar	Dizi
1	7	7, 11, 15, 3, 16, 13
2	15	7, 11, 15, 3, 16, 13
3	3	3, 7, 11, 15, 16, 13
4	16	3, 7, 11, 15, 16, 13
5	13	3, 7, 11, 13, 15, 16

- `while` döngüsünün tekrar sayısı j'ye bağlıdır. En kötü durumda toplam $1+2+\cdots+(n-1)$ olur.
- En kötü ve ortalama durum $O(n^2)$. En iyi durum (dizi zaten sıralı) $O(n)$.

Seçmeli sıralama: Dış döngü $n-1$ kez döner. İç döngü her turda bir azalarak döner. - Karşılaştırma sayısı $(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}$. - En iyi, en kötü ve ortalama durumda $O(n^2)$.

Ders sonrası notlar

- **Prefix değerlendirmede ara sonuç:** Hoca derste "20 ile 450'yi çarptı, 900" dedi. $20\cdot 450=9000$'dir. Slaytta da 9000 yazıyor. Sonra $9000/300=30$ ve $30-2=28$ olur. Hoca da sonucu 28 olarak doğru söyledi.
- **Prefix/postfix sürçmesi:** Hoca $(A+B)*(C/D)$ örneğinde prefix yazarken bir an "postfix ise önüne çarpı gelecek" dedi. Prefix'te işaret öne, postfix'te sona gelir.
- **İlk örneğin ifadesi:** Altyazıda "A artı B çarpı C eksi D" geçiyor. Tahtadaki ifade $(A+B)*(C/D)$'dir. Özetteki dönüşümler tahtadaki ifadeye göredir.
- **Kuyruk için LILO:** Hoca kuyruğu "FIFO ve LILO" diye adlandırdı. İkisi aynı davranışı anlatır. Literatürde yaygın kullanılan ad FIFO'dur.
- **Yığın göstergesinin yönü:** Hoca her push'ta SP'nin arttığını anlattı. Bu, yığının yukarı doğru büyüdüğü bir modelde doğrudur. x86 gibi birçok mimaride yığın aşağı doğru büyür; push'ta SP azalır, pop'ta artar. Alt Seviye Programlama dersinde bu ayrıntı önemlidir.
- **Heap bir yığın değildir:** Hoca "heap de dinamik olarak oluşturulmuş bir stack'tir" dedi. Veri yapısı olarak heap, her düğümün çocuklarından büyük (veya küçük) olduğu tam ikili ağaçtır. Bellekteki "heap" bölgesi ise dinamik bellek ayırma alanıdır ve yığından (stack) ayrı bir bölgedir.
- **Big-O, Big-Omega ve Big-Theta durum değildir:** Hoca Big-O'yu "en kötü durum", Big-Omega'yı "en iyi durum", Big-Theta'yı "ortalama durum" diye eşledi. Doğrusu: Bu üçü fonksiyonlar için **üst sınır, alt sınır ve sıkı sınır** gösterimleridir. En kötü, en iyi ve ortalama durum ise ayrı kavramlardır. Her durumun kendi O, Ω ve Θ'sı olabilir. Örnek: araya ekleme sıralamasının en kötü durumu $\Theta(n^2)$, en iyi durumu $\Theta(n)$'dir.
- **$k=2$ ispatındaki sürçme:** Hoca "x 2'den büyükse $2x$ de x^2'den büyüktür" dedi. Doğrusu tersidir: $x>2$ ise $x^2>2x$. İspatta da $2x$ yerine x^2 konarak bu doğru yön kullanıldı.
- **Limit yöntemi:** Hoca limitin "x sıfıra giderken" alındığını söyledi. Limit $x\to\infty$ iken alınır. Ayrıca limit 0 çıkınca $f(x)=O(g(x))$ olur, ama bu sıkı bir sınır değildir. Hoca "f'nin karmaşıklığı x^4'tür" dedi. $4x^3+3x^2$ fonksiyonunun sıkı sınırı $\Theta(x^3)$'tür; x^4 sadece bir üst sınırdır.
- **$\log(n!)$:** Hoca "$\log n!$, n^n'dir" dedi. Doğrusu $n!\le n^n$ olduğu için $\log(n!)\le n\log n$ eşitsizliğidir. Bu, $O(n\log n)$ sonucu için yeterlidir. Aslında $\log(n!)=\Theta(n\log n)$'dir.
- **Seçmeli sıralamada tam sayım:** Dış döngü $i=0,\ldots,n-2$ olmak üzere $n-1$ kez döner. İç döngü toplamı tam olarak $\frac{n(n+1)}{2}-1$'dir. Slayttaki $\frac{n(n+1)}{2}$ bir üst yaklaşımdır. Karmaşıklık değişmez: $O(n^2)$. Ayrıca iç döngünün $j=i+1$'den başlaması yeterlidir; $A[i]$'yi kendisiyle karşılaştırmaya gerek yoktur.
- **Infix değerlendirmede eşit öncelik:** Slayttaki algoritma $/$ gelince yığındaki $*$'ı yapmadan $/$'yı üstüne koyuyor. Sola birleşimli işlemlerde standart kural, eşit öncelikteki işlemi önce yapmaktır: $(2\cdot 45)/15$. Bu örnekte iki yol da 6 verir. Ama tam sayı bölmesinde sonuç değişebilir.
- **Araya ekleme sıralaması en iyi durum:** Hoca en iyi durumu "ilk birinci de bulmuş" diye anlattı. Doğrusu: dizi zaten sıralıysa her eleman tek karşılaştırmayla yerinde kalır. Toplam $n-1$ karşılaştırma, $O(n)$.

- **Quicksort:** Hoca quicksort'u $O(n \log n)$ olarak verdi. Bu ortalama durumdur. Quicksort'un en kötü durumu $O(n^2)$'dir.
- **Shell sort:** Hoca shell sort'u $O(n^2)$ algoritmalar arasında saydı. Shell sort'un karmaşıklığı seçilen aralık dizisine bağlıdır. Örneğin bazı dizilerle $O(n^{3/2})$ veya $O(n \log^2 n)$ olur.
- **Kübik örnek:** Hoca "belki 3 tane matrisin çarpımı" dedi. Klasik yöntemle iki $n \times n$ matrisin çarpımı zaten $O(n^3)$'tür.
- **`BulEnkucuk` parametresi:** Slayttaki fonksiyon `BulEnkucuk(float A[])` biçimindedir ve n parametre olarak verilmemiştir. Kodun derlenebilmesi için n de parametre olarak (`int n`) eklenmelidir.
- **Hanoi kuleleri:** n diskte en az $2^n - 1$ hamle gerekir. Karmaşıklık $O(2^n)$'dir.
- Bütün sayısal örnekler (31, 4, 28, 16, $4n + 6$, $4n$, $2n^2 + 9n + 2$, $4n^2 + 4n + 2$, $3^7 < 7!$, araya ekleme adımları) Python ile kontrol edildi.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.